

Design of a Clock Domain Crossing Circuit for Safe Pulse Transfer

By:

Suhani Pareek

Mahi Kumari

Electronics and Communication Engineering

Introduction

The reliable transfer of digital signals across asynchronous clock domains remains one of the most fundamental and critical challenges in modern digital system design. Clock Domain Crossing (CDC) introduces inherent risks such as metastability, pulse loss, and data corruption, phenomena that, if left unaddressed, can severely compromise the integrity and functionality of a digital circuit.

This project aims to systematically address these challenges through the design and functional verification of a CDC circuit capable of safely transferring a single cycle pulse from a source clock domain to a destination clock domain. By exploring synchronization techniques and applying rigorous RTL design principles, this work seeks to demonstrate a reliable and efficient solution to one of the most well known problems in synchronous digital design.

Objectives

- Understand the root causes of Clock Domain Crossing (CDC) issues such as metastability, pulse loss, and data corruption.
 - Study and apply reliable synchronization techniques for safe pulse transfer across asynchronous clock domains.
 - Design an RTL level circuit that delivers a clean, exactly one cycle wide pulse at the destination every single time.
 - Verify the design rigorously using a testbench with different clock frequencies and random pulse scenarios.
-

Github: https://github.com/zaimon3333/Xenith_Core

Features

- **Dual Independent Clock Support** — The design operates with two completely independent clocks running at different frequencies (10ns and 17ns), simulating a real world asynchronous environment.
 - **Toggle-Based Pulse Stretching** — A toggle flip-flop in the source domain stretches the incoming single-cycle pulse, ensuring it is wide enough to be reliably caught by the destination domain.
 - **Two Flip-Flop Synchronizer** — A classic and proven two stage synchronizer bridges the clock domains, effectively eliminating metastability before it can propagate.
 - **Clean Pulse Generation** — An XOR-based edge detector at the output produces a crisp, exactly one cycle wide pulse in the destination domain, nothing more, nothing less.
 - **Zero Pulse Loss & No Duplication** — Every pulse that enters the source domain makes it safely to the destination, with no missed pulses and no unwanted extras.
 - **Reset Support** — Both source and destination domains have independent reset signals, ensuring the design initializes cleanly every time.
 - **Random Pulse Testing** — The testbench generates pulses at random intervals using `$urandom_range`, stress-testing the design under unpredictable real-world conditions.
 - **Real-Time Monitoring** — `$monitor` continuously tracks and displays pulse activity across both domains throughout the simulation.
-

Architecture(Schematic) Overview

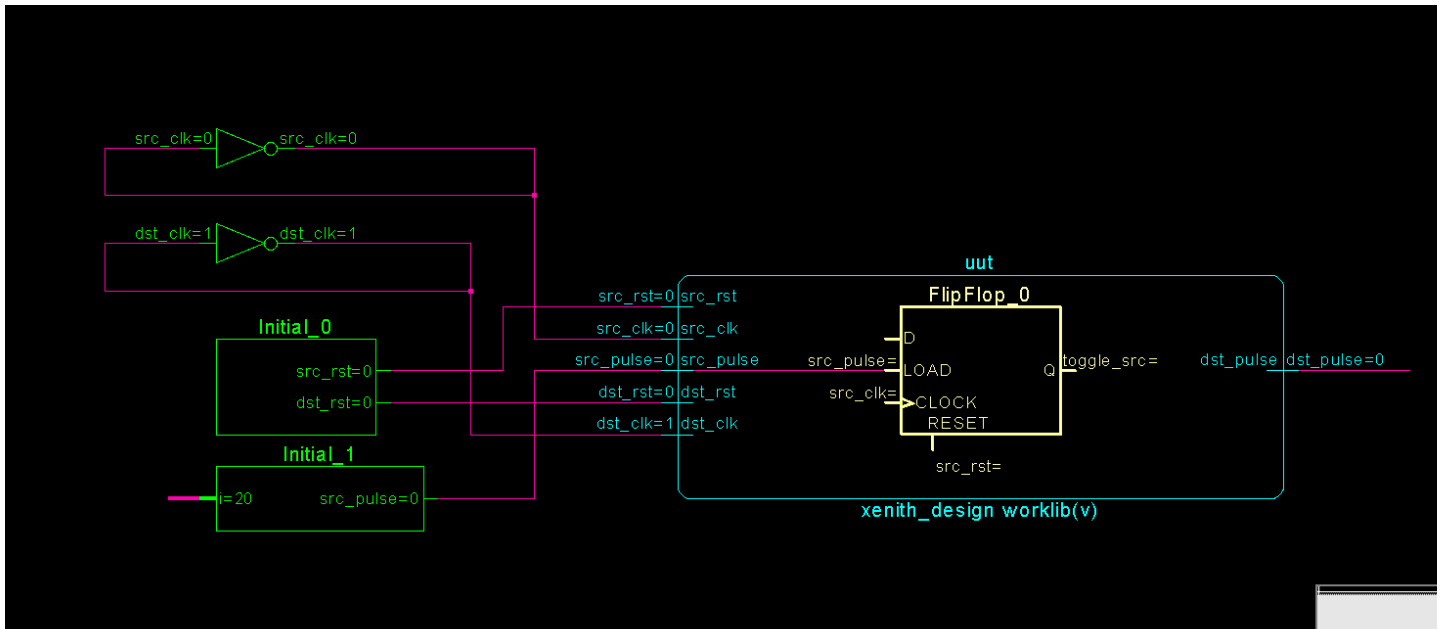
The design is built around a clean and elegant three-stage architecture, where each block has one job — and does it well.

- **Toggle Flip-Flop (Source Domain)** — When a pulse arrives on `src_pulse`, instead of passing it directly, the circuit toggles a signal called `toggle_src`. This is the secret sauce, converting a short pulse into a level change that won't get lost, no matter how different the two clock speeds are.
- **Two Flip-Flop Synchronizer (CDC Bridge)** — The `toggle_src` signal is passed through two back to back flip-flops (`sync_ff1` and `sync_ff2`), both running on the

destination clock. This is the industry-standard way of safely crossing a clock boundary, it gives metastability no chance to survive.

- **Edge Detector (Destination Domain)** — Once synchronized, the design simply XORs `sync_ff1` and `sync_ff2`. Every time `toggle_src` changes state, this XOR fires: producing a clean, exactly one-cycle-wide pulse on `dst_pulse`, right on cue.

Both domains have independent reset logic (`src_rst` and `dst_rst`), keeping everything clean and controlled from the start.



Design

The design is implemented in Verilog and built around a simple but effective toggle-based CDC technique. Here's how it works:

- **Toggle Flip-Flop (Source Domain)** — Every time a pulse arrives on `src_pulse`, a toggle register flips its state. This converts the short pulse into a level change that's easy to detect across clock boundaries.
- **Two Flip-Flop Synchronizer (Destination Domain)** — The toggled signal passes through two back to back flip-flops (`sync_ff1` and `sync_ff2`), both clocked by `dst_clk`. This safely resolves any metastability before the signal is used.
- **Pulse Generation (XOR Logic)** — A simple XOR between `sync_ff1` and `sync_ff2` detects the transition and fires a clean, exactly one-cycle-wide pulse (`dst_pulse`) in the destination domain.

- **Reset Logic** — Both domains support independent active high resets (`src_rst` and `dst_rst`), ensuring a clean startup every time.



```
1 module xenith_design(
2   input wire src_rst,
3   input wire src_clk,
4   input wire src_pulse,
5   input wire dst_rst,
6   input wire dst_clk,
7   output wire dst_pulse
8 );
9
10 reg toggle_src;
11
12 always @(posedge src_clk or posedge src_rst) begin
13   if(src_rst)
14     toggle_src <= 0;
15   else if(src_pulse)
16     toggle_src <= ~toggle_src;
17   end
18
19 reg sync_ff1, sync_ff2;
20
21 always @(posedge dst_clk or posedge dst_rst) begin
22   if(dst_rst) begin
23     sync_ff1 <= 0;
24     sync_ff2 <= 0; end
25   else begin
26     sync_ff1 <= toggle_src;
27     sync_ff2 <= sync_ff1;
28   end end
29
30 assign dst_pulse = sync_ff1 ^ sync_ff2;
31 endmodule
```

Validation (Testbench)

The testbench was written to simulate real-world conditions as closely as possible. Two independent clocks are generated, `src_clk` toggling every **5ns** (100MHz) and `dst_clk` toggling every **8.5ns** (~59MHz), mimicking two completely asynchronous clock domains.

At the start, both resets are asserted to initialize the design cleanly. Then, **20 random pulses** are fired at the source using `$urandom_range`, each held high for 10ns before being released, keeping things unpredictable and realistic.

A `$monitor` statement continuously watches and prints the `src_pulse` and `dst_pulse` signals in real time, making it easy to verify that every pulse sent in is safely received on the other side.

Open  xenith_testbench.v ~\Xenith Core Save

xenith_design.v x xenith_testbench.v

```
1 module tb;
2
3 reg src_clk = 0 ;
4 reg dst_clk = 0 ;
5 reg src_rst = 1 ;
6 reg dst_rst = 1 ;
7 reg src_pulse = 0 ;
8 wire dst_pulse;
9
10 xenith_design uut(.src_rst(src_rst), .src_clk(src_clk), .dst_rst(dst_rst), .dst_clk(dst_clk), .dst_pulse(dst_pulse), .src_pulse(src_pulse));
11
12 always #5 src_clk=~src_clk ;
13 always #8.5 dst_clk=~dst_clk ;
14
15 initial begin
16 #20;
17 $display("----Simulation is Starting----");
18 src_rst = 0;
19 dst_rst = 0;
20 end
21
22 integer i;
23 initial begin
24 #30;
25 $display("----Generating Random Pulses----");
26 for(i=0;i<20;i=i+1) begin
27 #($random_range(0,50));
28 src_pulse = 1;
29 #10;
30 src_pulse = 0;
31 end end
32
33 initial begin
34 $monitor("Time=%0t  src_pulse=%b  dst_pulse=%b", $time, src_pulse, dst_pulse);
35 end
36
37 initial begin
38 #1000;
39 $display("----Simulation is Ending----");
40 $finish;
41 end
42 endmodule
```

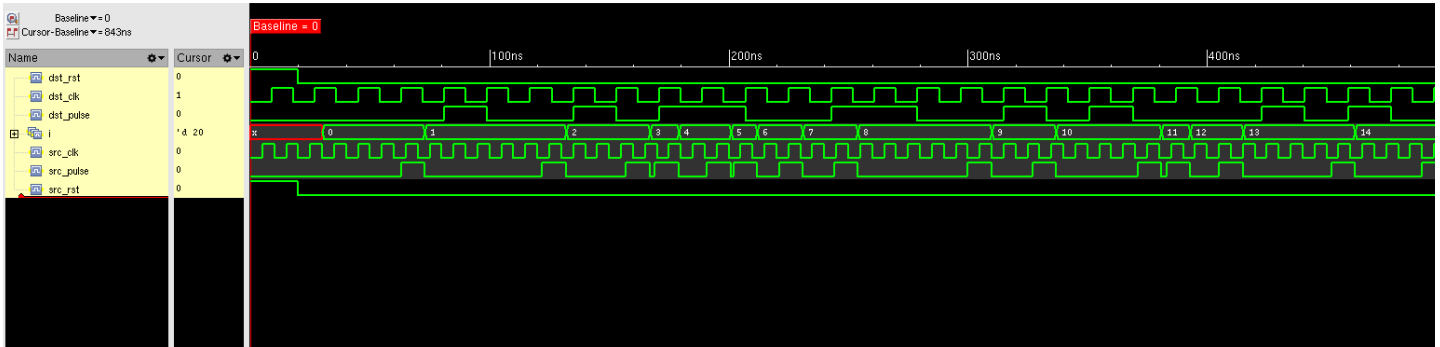
Waveform Analysis

The simulation was run on Cadence SimVision over a total time of 1000ns. Here's what the waveform tells us:

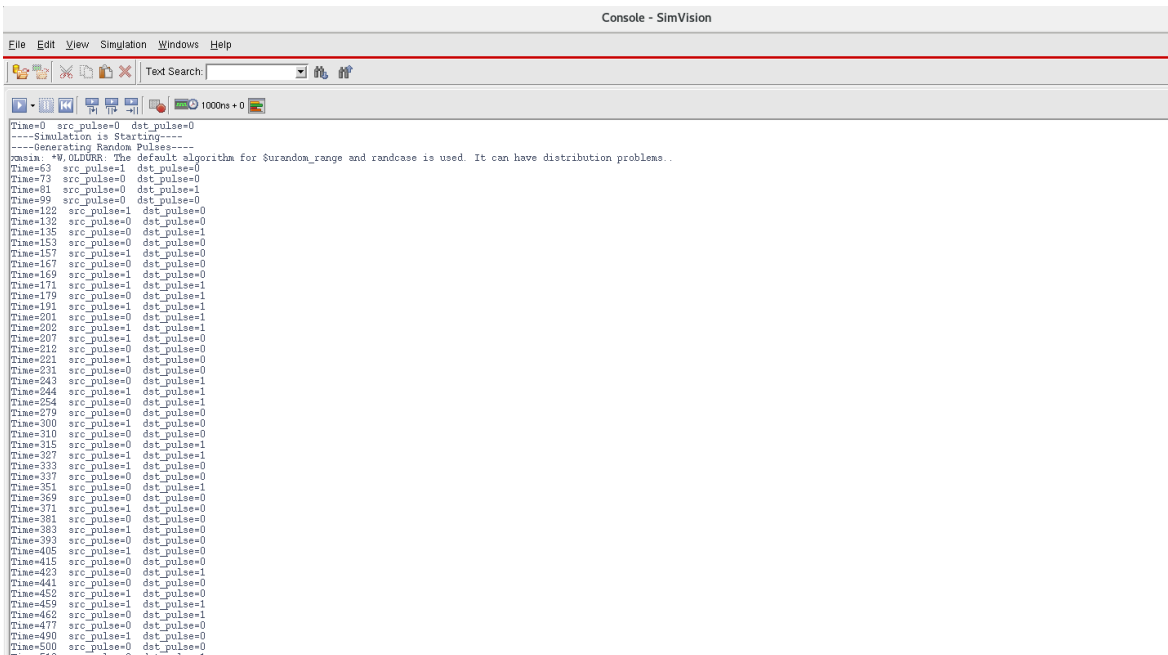
- **src_clk & dst_clk** — Both clocks are clearly visible running at different frequencies throughout the simulation, confirming the two independent asynchronous clock domains are working correctly.
- **src_rst & dst_rst** — Both reset signals go low early in the simulation (around 20-30ns), releasing the circuit into normal operation as expected.
- **src_pulse** — Random pulses appear at irregular intervals across the simulation window, successfully mimicking real-world unpredictable input behaviour across 20 iterations.
- **dst_pulse** — Clean output pulses appear on the destination side corresponding to every source pulse, confirming that no pulse was lost or duplicated during the crossing.

- **Loop counter `i`** — Increments from 0 to 14 (visible in the waveform), tracking each pulse generation cycle cleanly.

The waveform visually confirms that the CDC circuit performs exactly as designed, every pulse makes it across the clock boundary safely, cleanly, and without corruption.



Output:



Synthesis Report Analysis

The design was synthesized using **Cadence Genus (TM) Synthesis Solution 21.14** under slow operating conditions. Here's what each report reveals:

Area Report:

- The entire design uses just **7 cells** with a total area of **80.988 μm^2**
- Net area is 0, confirming a clean, compact implementation with no unnecessary routing overhead.

```
1 =====
2  Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
3  Generated on:      May 05 2026  02:21:24 pm
4  Module:           xenith_design
5  Technology library: slow
6  Operating conditions: slow (balanced_tree)
7  Wireload mode:     enclosed
8  Area mode:         timing library
9 =====
10
11  Instance  Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
12  -----
13  xenith_design      7      80.988      0.000      80.988 <none> (D)
14  (D) = wireload is default in technology library
```

Gate Level Report:

The 7 cells break down as:

- **SDFFRHQX1** (1 instance) — the toggle flip-flop in source domain
- **DFFRHQX1** (2 instances) — the two synchronizer flip-flops
- **CLKXOR2X1** (1 instance) — pulse detection XOR gate
- **INVXL** (3 instances) — inverters for reset logic

Sequentials dominate at **81.3%** of total area, which makes perfect sense for a synchronizer-heavy design.

```
1 =====
2  Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
3  Generated on:      May 05 2026  02:21:36 pm
4  Module:           xenith_design
5  Technology library: slow
6  Operating conditions: slow (balanced_tree)
7  Wireload mode:     enclosed
8  Area mode:         timing library
9 =====
10
11
12  Gate      Instances  Area  Library
13  -----
14  CLKXOR2X1      1  8.326  slow
15  DFFRHQX1       2 40.873  slow
16  INVXL          3  6.812  slow
17  SDFFRHQX1      1 24.978  slow
18  -----
19  total          7 80.988
20
21
22
23  Type      Instances  Area  Area %
24  -----
25  sequential      3 65.850  81.3
26  inverter        3  6.812   8.4
27  logic           1  8.326  10.3
28  physical_cells   0  0.000   0.0
29  -----
30  total          7 80.988 100.0
31
```


Power Report:

- Total power consumption: **~1.74 μ W** — extremely low.
- Registers consume the most at **88.44%**, again expected given the flip-flop dominated design.
- Leakage power is just **25.98%** of total, with internal switching making up the rest.

```
1 Instance: /xenith_design
2 Power Unit: W
3 PDB Frames: /stim#0/frame#0
4 -----
5 Category          Leakage      Internal      Switching      Total      Row%
6 -----
7 memory            0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
8 register          3.53526e-07  1.08659e-06  9.96300e-08  1.53975e-06  88.44%
9 latch             0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
10 logic            9.88136e-08  9.10312e-08  1.13400e-08  2.01185e-07  11.56%
11 bbox             0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
12 clock            0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
13 pad              0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
14 pm               0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
15 -----
16 Subtotal         4.52340e-07  1.17762e-06  1.10970e-07  1.74093e-06  100.00%
17 Percentage        25.98%      67.64%      6.37%      100.00%  100.00%
18 -----
```

Timing Report:

- The timing report notes that some paths are unconstrained and have not been fully displayed.
- This is expected behavior for a CDC design, since the two clocks are asynchronous, traditional static timing analysis cannot fully analyze cross-domain paths.
- This is precisely why functional simulation and synchronizer based design techniques are used to ensure reliability instead.

```
1 =====
2 Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
3 Generated on:      May 05 2026 02:23:10 pm
4 Module:           xenith_design
5 Operating conditions: slow (balanced_tree)
6 Wireload mode:    enclosed
7 Area mode:        timing library
8 =====
9
10 Some unconstrained paths have not been displayed.
11 Use -unconstrained or set the root attribute 'timing_report_unconstrained' to 'true' to see only these unconstrained paths.
12
```

Future Scope:

Since CDC is a deterministic digital design problem, so ML won't replace synchronizer. However, we can smartly add ML around project to make it stand out in.

1. Pulse Pattern Prediction Using Machine Learning

While the current design ensures safe pulse transfer reactively, an intelligent extension would be to make the system *predictive*. By training a machine learning model — such as an LSTM (Long Short-Term Memory) network — on historical pulse timing data collected from the source domain, the system can learn to anticipate when the next pulse is likely to arrive and at what frequency.

The model takes pulse timestamps and inter-arrival intervals as input features and predicts the next pulse window with reasonable accuracy. This prediction can then be used to:

- Enable **adaptive buffering** — pre-allocating resources in the destination domain just before the pulse arrives, reducing latency.
- Drive **dynamic clock scaling** — temporarily boosting the destination clock frequency during predicted burst periods to ensure no pulse is missed, and scaling it back during idle windows to save power.

This transforms the CDC circuit from a passive relay into an intelligent, self-optimizing data path — a concept directly applicable in high-speed SoC design and network-on-chip (NoC) architectures.

2. CDC Failure Detection Using Machine Learning

One of the most impressive and industrially relevant extensions is using ML for real-time CDC failure detection. In complex chips with hundreds of clock domain crossings, manually identifying metastability events, missing pulses, or glitches is nearly impossible.

The proposed approach involves:

- **Data Collection** — Extracting waveform data from simulation or silicon debug tools, capturing signals like `src_pulse`, `toggle_src`, `sync_ff1`, `sync_ff2`, and `dst_pulse` over time.
- **Feature Engineering** — Deriving features such as pulse width, toggle frequency, synchronizer settling time, and clock edge relationships.
- **Model Training** — Training a classification model (e.g., Random Forest or CNN on waveform images) to distinguish between three classes: **Normal operation**, **CDC error (pulse loss/duplication)**, and **Metastability risk**.
- **Real-time Inference** — Deploying the trained model as a monitoring layer that flags anomalies during simulation or post-silicon validation.

This approach is already being explored in advanced EDA research and silicon debug flows, where catching a single CDC failure early can save millions in re-spins. Integrating ML-based CDC monitoring into verification pipelines represents the next frontier in robust digital design.
